

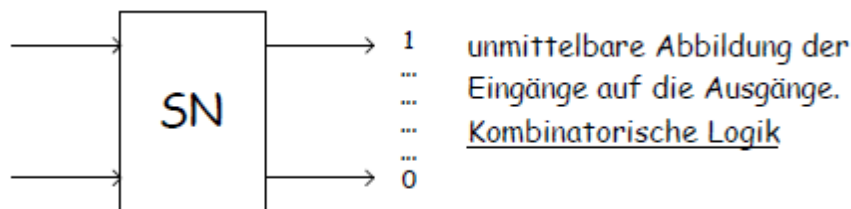
4. Schaltnetze

4.1 Begriffe

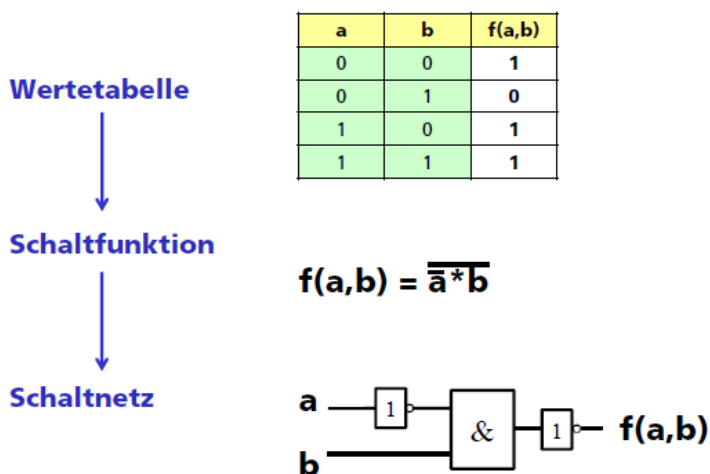
Definition (Schaltnetz SN):

Ein Schaltnetz (oder kombinatorische Logik, engl. combinational/combinatorial logic) ist eine digitale Schaltung, in der es für jede mögliche Kombination der Eingangssignale (Eingangsbelegung) nur eine mögliche Ausgangskombination gibt.

Die Eingänge werden also unmittelbar auf die Ausgänge abbildet, es wird nichts aus der Vergangenheit gespeichert, was das Ergebnis noch beeinflussen könnte.



Bei der Schaltungssynthese wird für eine gegebene Schaltfunktion (Wahrheitstabelle, schaltalgebraischer Ausdruck) eine Digitalschaltung gesucht, die diese Schaltfunktion realisiert:



Diese Schaltung soll bezüglich bestimmter Vorgaben (z. Bsp. Durchlaufzeit, Ressourcenbedarf) optimal sein.

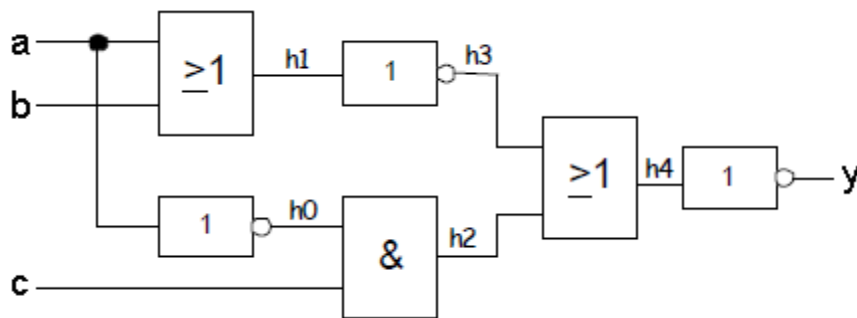
Ist eine Schaltung gegeben und wird hierfür ein schaltalgebraischer Ausdruck (und die Wahrheitstabelle) gesucht, so spricht man von einer Schaltnetzanalyse.

4.2 Analyse von Schaltnetzen

Vorgehen bei der Analyse von Schaltnetzen:

1. Hilfsvariablen einführen
2. Ausdruck mit Hilfe der Hilfsvariablen angeben
3. Hilfsvariablen sukzessiv ersetzen
4. Umformen und vereinfachen des Ausdrucks
5. Wahrheitstabelle aufstellen

Beispiel:



Schaltalgebraischer Ausdruck:

$$\begin{aligned}
 y &= \overline{h_4} = \overline{h_3 \vee h_2} = \overline{h_3} \wedge \overline{h_2} = \overline{a \vee b} \wedge \overline{h_0 \wedge c} \\
 &= (\overline{a \vee b}) \wedge (\overline{h_0} \wedge \overline{c}) \\
 &= (\overline{a \vee b}) \wedge (a \vee \overline{c}) \\
 &= a \wedge \overline{a \vee b} \vee ab \wedge \overline{c} \\
 &= a \wedge \overline{a} \wedge \overline{b} \vee ab \wedge \overline{c} \\
 &= 0 \vee ab \wedge \overline{c} \\
 &= ab \wedge \overline{c}
 \end{aligned}$$

Wahrheitstabelle: $y = a \vee b\overline{c}$

c	b	a	$\overline{c}$	$b\overline{c}$	y
0	0	0	1	0	0
0	0	1	1	0	1
0	1	0	1	1	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	0	0	1
1	1	0	0	0	0
1	1	1	0	0	1

4.3 Produktterme und Summenterme

Definition (Belegungsblock):

Gegeben sei eine SF mit n Variablen. Mehrere Belegungen bilden eine Belegungsblock, wenn sie an m Stellen ($m \leq n$) übereinstimmen und für die restlichen $n-m$ Stellen alle Kombinationen vorhanden sind. Diese Stellen werden als Freistellen (Kennzeichnung: don't care -) bezeichnet.

Beispiel:

(0,1,1,-) (0,1,1,0)
 (0,1,1,1)

(-, -, 0, 1) (0 0 0 1)
 (0 1 0 1)
 (1 0 0 1)
 (1 1 0 1)

Satz:

Ein Belegungsblock mit k Freistellen umfasst 2^k Einzelbelegungen.

Eine einzelne Belegung kann auch aufgefasst werden als Belegungsblock mit 0 Freistellen.

Definition (Produktterm, Summenterm):

Gegeben sei eine SF mit n Variablen. Eine Konjunktion (Disjunktion) von m Variablen ($m \leq n$), bei der jede Variable **höchstens einmal** vorkommt, heißt Produktterm (Summenterm).

Satz:

Ein Produktterm (Summenterm), bei dem $k=n-m$ Variablen fehlen, beschreibt 2^k Einsstellen (Nullstellen). Diese liegen in einem Belegungsblock.

Beispiel für Produktterme:

$f(x_4, x_3, x_2, x_1)$

Produktterm	Belegungsblock der Einsstellen	Auflistung der Einsstellen
$x_4 \cdot \bar{x}_3 \cdot x_2 \cdot x_1$	(1 0 1 1)	B ₁₁
$x_4 \cdot \bar{x}_3 \cdot x_1$	(1 0 - 1)	B ₉ , B ₁₁
$x_4 \cdot \bar{x}_2$	(1 - 0 -)	B ₈ , B ₉ , B ₁₂ , B ₁₃
$\bar{x}_2$	(- - 0 -)	B ₀ , B ₁ , B ₄ , B ₅ , B ₈ , B ₉ , B ₁₂ , B ₁₃

Beispiel für Summenterme:

$$f(x_4, x_3, x_2, x_1)$$

Summenterm	Belegungsblock der Nullstellen	Auflistung der Nullstellen
$\bar{x}_2$	(--1-)	B ₂ , B ₃ , B ₆ , B ₇ , B ₁₀ , B ₁₁ , B ₁₄ , B ₁₅
$x_3 \vee \bar{x}_1$	(-0-1)	B ₁ , B ₃ , B ₉ , B ₁₁
$x_3 \vee \bar{x}_2 \vee \bar{x}_1$	(-011)	B ₃ , B ₁₁
$x_4 \vee \bar{x}_3 \vee \bar{x}_2 \vee x_1$	(0110)	B ₆

Definition (Minterm):

Eine Konjunktion, bei der alle n Variablen einer Schaltfunktion **genau einmal** vorkommen (entweder bejaht oder negiert), heißt Vollkonjunktion oder Minterm.

Ein Minterm ist ein Spezialfall eines Produktterms, bei dem 0 Variablen fehlen. Der zugehörige Belegungsblock umfasst eine einzige Einstelle.

Definition (Maxterm):

Eine Disjunktion, bei der alle n Variablen einer Schaltfunktion **genau einmal** vorkommen (entweder bejaht oder negiert), heißt Volldisjunktion oder Maxterm.

Ein Maxterm ist ein Spezialfall eines Summenterms, bei dem 0 Variablen fehlen. Der zugehörige Belegungsblock umfasst eine einzige Nullstelle.

Satz:

Eine Schaltfunktion (SF), die durch einen einzigen Minterm beschrieben wird, besitzt genau eine Einsstelle.

Satz:

Eine Schaltfunktion (SF), die durch einen einzigen Maxterm beschrieben wird, besitzt genau eine Nullstelle.

Bezeichnungen:

Ein Minterm wird mit m_i bezeichnet, wenn i der Belegungsindex der zugehörigen Einsstelle ist,

Ein Maxterm wird mit M_i bezeichnet, wenn i der Belegungsindex der zugehörigen Nullstelle ist.

Satz:

Es gilt stets:

$$m_i = \overline{M_i}$$

Beispiel:

Gegeben seien 4 Schaltvariablen x_4, x_3, x_2, x_1 :

$$m_{11} = \overline{x_4} \overline{x_3} x_2 x_1 \qquad m_{11} = \overline{M_{11}}$$

$$m_3 = \overline{x_4} \overline{x_3} x_2 x_1$$

$$M_{11} = \overline{x_4} \vee x_3 \vee \overline{x_2} \vee \overline{x_1}$$

$$M_8 = \overline{x_4} \vee x_3 \vee x_2 \vee x_1$$

4.4 Disjunktive und konjunktive Normalform

Definition (Disjunktive Normalform DNF):

Wird eine Schaltfunktion beschrieben durch die disjunktive Verknüpfung von Mintermen, so nennt man diese Darstellung der Schaltfunktion die disjunktive Normalform.

Definition (Konjunktive Normalform KNF):

Wird eine Schaltfunktion beschrieben durch die konjunktive Verknüpfung von Maxtermen, so nennt man diese Darstellung der Schaltfunktion die konjunktive Normalform.

Hauptsatz der Schaltungsgebra:

Jede Schaltfunktion $f(x_n, \dots, x_1)$ kann eindeutig durch einen Ausdruck aus disjunktiv verknüpften Mintermen oder durch einen Ausdruck aus konjunktiv verknüpften Maxtermen dargestellt werden. Für jede Schaltfunktion existiert also die Darstellung in der Disjunktiven Normalform und die Darstellung in der Konjunktiven Normalform.

Für die **DNF** gelten die folgenden Formeln:

$$f(x_n, \dots, x_1) = \bigvee_{\{i | B_i \in \mathcal{E}(f)\}} m_i$$

$\mathcal{E}(f)$ bezeichnet dabei die Menge der Einsstellen von f .

Das gleiche Ergebnis erhält man, wenn man alle Minterme mit dem Funktionswert konjunktiv verknüpft und die Ergebnisse miteinander disjunktiv verknüpft:

$$f(x_n, \dots, x_1) = \bigvee_{i=0}^{2^n-1} [m_i \cdot f(B_i)]$$

Für die **KNF** gelten die folgenden Formeln:

$$f(x_n, \dots, x_1) = \prod_{\{i | B_i \in \mathcal{N}(f)\}} M_i$$

$\mathcal{N}(f)$ bezeichnet dabei die Menge der Nullstellen von f .

Das gleiche Ergebnis erhält man, wenn man alle Maxterme mit dem Funktionswert disjunktiv verknüpft und die Ergebnisse miteinander konjunktiv verknüpft:

$$f(x_n, \dots, x_1) = \prod_{i=0}^{2^n-1} [M_i \vee f(B_i)]$$

Beispiel:

c	b	a	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

$$\text{DNF: } f(c,b,a) = m_0 f(B_0) \vee m_1 f(B_1) \vee m_2 f(B_2) \vee m_3 f(B_3) \vee m_4 f(B_4) \vee m_5 f(B_5) \vee m_6 f(B_6) \vee m_7 f(B_7)$$

$$= m_1 \vee m_2 \vee m_4 \vee m_7$$

$$= \bar{c} \cdot \bar{b} \cdot a \vee \bar{c} \cdot b \cdot \bar{a} \vee c \cdot \bar{b} \cdot \bar{a} \vee c \cdot b \cdot a$$

$$\text{KNF: } f(c,b,a) = [M_0 \vee f(B_0)] \cdot [M_1 \vee f(B_1)] \cdot [M_2 \vee f(B_2)] \cdot [M_3 \vee f(B_3)] \cdot [M_4 \vee f(B_4)] \cdot [M_5 \vee f(B_5)] \cdot [M_6 \vee f(B_6)] \cdot [M_7 \vee f(B_7)]$$

$$= M_0 \cdot M_3 \cdot M_5 \cdot M_6$$

$$= [c \vee b \vee a] \cdot [c \vee \bar{b} \vee \bar{a}] \cdot [\bar{c} \vee b \vee \bar{a}] \cdot [\bar{c} \vee \bar{b} \vee a]$$

4.5 Primblöcke und Primterme

Definition (Einsblock, Implikant).

Gegeben sei eine Schaltfunktion f . Ein Belegungsblock, der nur Einsstellen von f enthält, heißt Einsblock von f .

Der zugehörige Produktterm heißt Implikant von f .

Definition (Nullblock, Implikat).

Gegeben sei eine Schaltfunktion f . Ein Belegungsblock, der nur Nullstellen von f enthält, heißt Nullblock von f .

Der zugehörige Summenterm hei Implikat von f .

Definition (Primeinsblock, Primimplikant PI).

Gegeben sei eine Schaltfunktion f . Ist ein Einsblock von f nicht mehr in einem größeren Einsblock von f enthalten, so wird er Primeinsblock von f genannt.

Der zugehörige Produktterm hei Primimplikant von f .

Definition (Primnullblock, Primimplikat PA).

Gegeben sei eine Schaltfunktion f mit n . Ist ein Nullblock von f nicht mehr in einem größeren Nullblock von f enthalten, so wird er Primnullblock von f genannt.

Der zugehörige Summenterm heißt Primimplikat von f .

Satz:

Jede Schaltfunktion ist als Disjunktion ihrer Primimplikanten oder als Konjunktion ihrer Primimplikate darstellbar.

KV-Diagramme:

Beispiel:

x_4	x_3	x_2	x_1	f	
0	0	0	0	0	
0	0	0	1	0	
0	0	1	0	0	
0	0	1	1	0	
0	1	0	0	1	B_4
0	1	0	1	0	
0	1	1	0	1	B_6
0	1	1	1	0	
1	0	0	0	0	
1	0	0	1	0	
1	0	1	0	0	
1	0	1	1	0	
1	1	0	0	1	B_{12}
1	1	0	1	0	
1	1	1	0	1	B_{14}
1	1	1	1	0	

In dieser Darstellung ist nicht erkennbar, dass die Einstellen B_4 , B_6 , B_{12} und B_{14} einen Belegungsblock bilden und daher mit einem einzigen Produktterm beschrieben werden können.

Abhilfe schafft hier das Diagramm nach Karnaugh – Veith (KV – Diagramm):

	x_1			
	0 ₀	0 ₁	0 ₅	1 ₄
x_2	0 ₂	0 ₃	0 ₇	1 ₆
	0 ₁₀	0 ₁₁	0 ₁₅	1 ₁₄
	0 ₈	0 ₉	0 ₁₃	1 ₁₂
	x_3			

$$f(x_4, x_3, x_2, x_1) = x_3 \cdot \bar{x}_1$$

KV-Diagramme für Schaltfunktionen bis zu 6 Schaltvariablen:

KV-Diagramm für 2 Schaltvariablen:

	x_1	
	0	1
x_2	2	3

KV-Diagramm für 3 Schaltvariablen:

	x_1			
	0	1	5	4
x_2	2	3	7	6
	x_3			

KV-Diagramm für 4 Schaltvariablen:

	x_1			
	0	1	5	4
	2	3	7	6
x_2	10	11	15	14
	8	9	13	12
	x_3			
	x_4			

KV-Diagramm für 5 Schaltvariablen:

	x_1				x_1			
	0	1	5	4	20	21	17	16
	2	3	7	6	22	23	19	18
x_2	10	11	15	14	30	31	27	26
	8	9	13	12	28	29	25	24
	x_3				x_5			
	x_4				x_4			

KV-Diagramm für 6 Schaltvariablen:

	x_1				x_1			
	0	1	5	4	20	21	17	16
	2	3	7	6	22	23	19	18
x_2	10	11	15	14	30	31	27	26
	8	9	13	12	28	29	25	24
	x_3				x_5			
	x_4				x_4			
	40	41	45	44	60	61	57	56
x_2	42	43	47	46	62	63	59	58
	34	35	39	38	54	55	51	50
	32	33	37	36	52	53	49	48
	x_3				x_5			
	x_6				x_6			

4.6 Unvollständige Schaltfunktionen

Definition (Unvollständige Schaltfunktion):

Ist bei einer Schaltfunktion nicht für alle Eingangsbelegungen ein Funktionswert definiert, so heißt sie unvollständige Schaltfunktion.

Definition (Definitionsbereich):

Belegungen einer unvollständigen Schaltfunktion, für die ein Funktionswert definiert ist, bilden den Definitionsbereich der unvollständigen Schaltfunktion.

Definition (Redundanz):

Belegungen einer unvollständigen Schaltfunktion, für die kein Funktionswert definiert ist, heißen Redundanzen, Freistellen oder "Don't-care-Belegungen". Statt dem Funktionswert wird bei den Redundanzen ein "-" oder ein "x" notiert.

Die Freistellen einer unvollständigen Schaltfunktion sind nicht mit den Freistellen in einem Belegungsblock zu verwechseln!

Definition (Vervollständigung):

Durch die Festlegung von Funktionswerten für die Redundanzen einer unvollständigen Schaltfunktion erhält man eine Vervollständigung der unvollständigen Schaltfunktion.

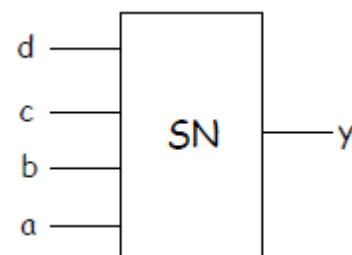
In der Praxis wird immer eine der möglichen Vervollständigungen realisiert.

Satz:

Eine unvollständige Schaltfunktion mit k Redundanzen besitzt 2^k Vervollständigungen.

Beispiel:

Gesucht sei ein SN das entscheidet, ob eine im 8-4-2-1-Code kodierte Ziffer ≥ 7 ist.



d	c	b	a	y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	-
1	0	1	1	-
1	1	0	0	-
1	1	0	1	-
1	1	1	0	-
1	1	1	1	-

a			
0 ₀	0 ₁	0 ₅	0 ₄
0 ₂	0 ₃	1 ₇	0 ₆
- ₁₀	- ₁₁	- ₁₅	- ₁₄
1 ₈	1 ₉	- ₁₃	- ₁₂
c			

Gesucht: möglichst einfache
disjunktive Form

$$\left. \begin{matrix} d \\ c b a \end{matrix} \right\} y = d \vee c b a$$

Bei der angegebenen Vervollständigung wird bei allen 6 Redundanzen der Funktionswert 1 realisiert.

Die im letzten Abschnitt eingeführten Begriffe „Primblöcke“ und „Primterme“ lassen sich wie folgt auf unvollständige Schaltfunktionen erweitern:

Definition (Implikanten, Einsblöcke, Primimplikanten, Primeinsblöcke von unvollständigen Schaltfunktionen):

Gegeben sei eine unvollständige SF f . Ein Produktterm heißt Implikant von f , wenn der zugehörige Belegungsblock **nur Einsstellen oder Redundanzen** von f umfasst.

Sind die Blöcke maximal groß, so heißen sie Primeinsblöcke von f , die zugehörigen Produktterme heißen dann Primimplikanten von f .

Definition (Implikate, Nullblöcke, Primimplikate, Primnullblöcke von unvollständigen Schaltfunktionen):

Gegeben sei eine unvollständige SF f . Ein Summenterm heißt Implikat von f , wenn der zugehörige Belegungsblock **nur Nullstellen oder Redundanzen** von f umfasst.

Sind die Blöcke maximal groß, so heißen sie Primnullblöcke von f , die zugehörigen Summenterme heißen dann Primimplikate von f .

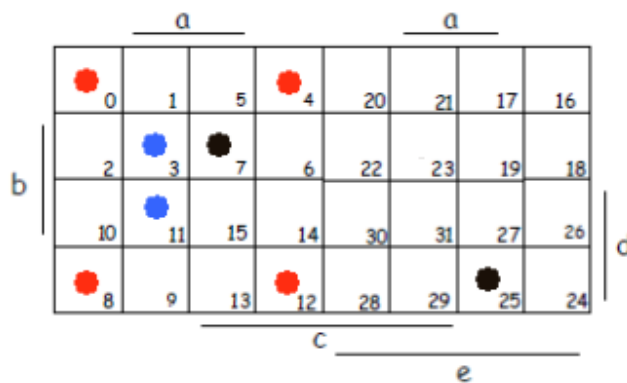
4.7 Bestimmung von Primimplikanten und Primimplikanten

4.7.1 Symmetrieverfahren im KV – Diagramm.

Hierbei handelt es sich um ein heuristisches Verfahren, d.h. es liegt kein strenger Algorithmus vor. Die Erfahrung des Anwenders ist erforderlich. Das Verfahren wird ab 6 Schaltvariablen sehr unübersichtlich.

Es muss immer überprüft, ob ein bereits gefundener Block noch in einem größeren Block enthalten ist. Dazu wird der Block an allen möglichen Symmetrielinien gespiegelt.

Die folgende Übung im KV-Diagramm für 5 SV soll dieses Vorgehen prinzipiell verdeutlichen:



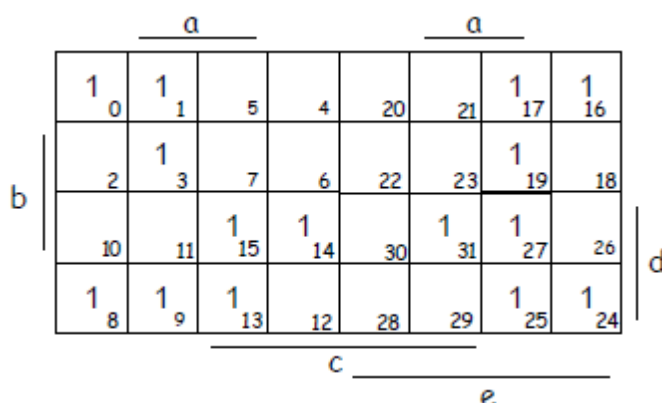
Betrachteter Block	Mögliche Erweiterungen zur doppelten Größe
Einserblock 7	3 5 6 15 23
Einserblock 25	27 29 24 9 17
Zweierblock 3,11	1,9 2,10 7,15 19,27
Viererbblock 0,4,8,12	20,28,16,24 2,10,6,14 1,5,9,13

Beispiel:

Gegeben sei SF $f(e,d,c,b,a)$

Einstellen: $E=\{0,1,3,8,9,13,14,15,16,17,19,24,25,27,31\}$

Redundanzen: $R=\{ \}$



$A: \bar{c} \bar{b}$
 $B: e \bar{c} a$
 $C: \bar{d} \bar{c} a$
 $D: \bar{e} d c a$
 $E: \bar{e} d \bar{b} a$
 $F: \bar{e} d c b$
 $G: d c b a$
 $H: e d b a$

Tipps für das Symmetrieverfahren im KV – Diagramm:

- Ein Block umfasst stets 2^n Felder.
- Redundanzen dürfen in die Blöcke aufgenommen werden.
- Eine Einsstelle (Nullstelle) kann in mehreren Blöcken enthalten sein.
- Selbst wenn alle Einsstellen (Nullstellen) durch Blöcke abgedeckt sind, kann es noch weitere maximal große Blöcke geben.

4.7.2 Verfahren nach Quine–McCluskey

Das Verfahren nach Quine und McCluskey (QMCV, nach Willard Van Orman Quine und Edward J. McCluskey) ist ein streng algorithmisches Verfahren, das sich daher auch programmieren lässt.

Der Kern des Verfahrens wurde bereits von Quine vollständig beschrieben. Die Verfeinerungen von McCluskey betreffen im Wesentlichen die praktische algorithmische Durchführbarkeit.

Das Verfahren geht aus von 1er-Blöcken und versucht systematisch, sie zu größeren Blöcken zu erweitern. Der folgende Satz macht eine Aussage darüber, ob zwei Blöcke der Größe 2^n sich zu einem Block der Größe 2^{n+1} zusammenfassen lassen:

Satz:

Zwei Belegungsblöcke der Länge n lassen sich zusammenfassen

- wenn sie an $n-1$ Stellen übereinstimmen und
- wenn an der nicht-übereinstimmenden Stelle der erste Block eine 1, der zweite Block eine 0 aufweist.

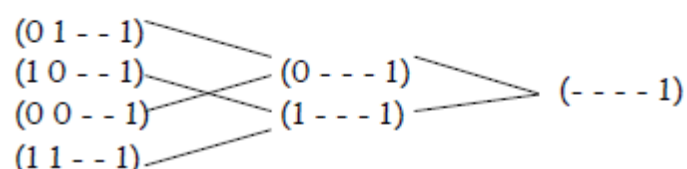
Der zusammengefasste Block besitzt an der nicht-übereinstimmenden Stelle eine Freistelle.

Beispiel:

$\left. \begin{array}{l} (0\ 1\ -\ -\ 1) \\ (0\ 0\ -\ -\ 1) \end{array} \right\} (0\ -\ -\ -\ 1)$

$\left. \begin{array}{l} (0\ 1\ -\ -\ 1) \\ (1\ 1\ -\ -\ 1) \end{array} \right\} (-\ 1\ -\ -\ 1)$

$\left. \begin{array}{l} (0\ 1\ -\ -\ 1) \\ (1\ 0\ -\ -\ 1) \end{array} \right\} \text{Nein}$



Mit der folgenden Definition erhält man einen weitem nützlichen Satz:

Definiton (Gewicht eines Belegungsblocks):

Das Gewicht eines Belegungsblocks ist die Anzahl der Stellen, die eine 1 aufweisen.

Satz:

Zwei zusammenfassbare Belegungsblöcke haben ein um eins verschiedenes Gewicht und die gleiche Anzahl von Redundanzen.

Damit kann das Verfahren von Quine-McClusky zur Bestimmung aller Primimplikanten / Primimplikate einer Schaltfunktion wie folgt formuliert werden:

1. Liste der Einzelbelegungen (Einsstellen und Redundanzen bzw. Nullstellen und Redundanzen) erstellen.
2. Die Einzelbelegungen (= Einserblöcke) werden, sortiert nach steigendem Gewicht G , in die erste Liste eingetragen.
3. Mit $G = 0$ beginnend werden nun alle Blöcke vom Gewicht G mit allen Blöcken vom Gewicht $G+1$ verglichen.
Falls Blöcke zusammenfassbar sind, werden diese gekennzeichnet und der dabei entstehende größere Block wird in eine neue Liste eingetragen.
4. Ende des Algorithmus, falls keine neue Liste entstanden ist.
Ansonsten wird Schritt 3 mit der neuen Liste wiederholt.
5. Die **nicht** gekennzeichneten Blöcke **aller** Listen sind die Belegungsblöcke, die zu den gesuchten Primimplikanten bzw. Primimplikaten gehören.

Anmerkungen:

1. Die bei Schritt 3 neu entstehenden Listen sind automatisch nach wachsendem Gewicht sortiert.
2. Das Mitführen der Belegungsindizes in den Listen kann die Übersichtlichkeit erhöhen.
3. Die Kennzeichnung bei Schritt 3 kann z. Bsp. durch einen Haken rechts neben dem Belegungsblock erfolgen. Ein Haken links kann dann benutzt werden, um zu kennzeichnen, dass alle nötigen Vergleiche durchgeführt wurden.
4. Primblöcke können nur aus Redundanzen bestehen (diese werden aber für die Darstellung der Funktion später nicht mehr benötigt).

Beispiel 1:Gegeben $f(d,c,b,a)$ $E=\{1,5,6,7,9,15\}$ $R=\{8,14\}$

Gesucht: alle PI

Unsortierte Liste:

i	Block	G
1	0 0 0 1	1
5	0 1 0 1	2
6	0 1 1 0	2
7	0 1 1 1	3
9	1 0 0 1	2
15	1 1 1 1	4
8	1 0 0 0	1
14	1 1 1 0	3

Listen der
1-er, 2-er
und 4-er
Blöcke:

i	Block
1	0 0 0 1 ✓
8	1 0 0 0 ✓
5	0 1 0 1 ✓
6	0 1 1 0 ✓
9	1 0 0 1 ✓
7	0 1 1 1 ✓
14	1 1 1 0 ✓
15	1 1 1 1 ✓

Ergebnis:

i	Block
1,5	0 - 0 1 (A)
1,9	- 0 0 1 (B)
8,9	1 0 0 - (C)
5,7	0 1 - 1 (D)
6,7	0 1 1 - ✓
6,14	- 1 1 0 ✓
7,15	- 1 1 1 ✓
14,15	1 1 1 - ✓

i	Block
6,7,14,15	- 1 1 - (E)

A: 0 - 0 1 $\bar{d} \bar{b} a$ B: - 0 0 1 $\bar{c} \bar{b} a$ C: 1 0 0 - $d \bar{c} \bar{b}$ D: 0 1 - 1 $\bar{d} c a$ E: - 1 1 - $c b$

Beispiel 2:

Gegeben: Vollständige Schaltfunktion $f(e,d,c,b,a)$ mit den Einsstellen

$E = \{0, 1, 3, 8, 9, 13, 14, 15, 16, 17, 19, 24, 25, 27, 31\}$

Gesucht: alle Primimplikanten.

i	Block
0	00000 ✓
1	00001 ✓
8	01000 ✓
16	10000 ✓
3	00011 ✓
9	01001 ✓
17	10001 ✓
24	11000 ✓
13	01101 ✓
14	01110 ✓
19	10011 ✓
25	11001 ✓
15	01111 ✓
27	11011 ✓
31	11111 ✓

i	Block
0,1	0000- ✓
0,8	0-000 ✓
0,16	-0000 ✓
1,3	000-1 ✓
1,9	0-001 ✓
1,17	-0001 ✓
8,9	0100- ✓
8,24	-1000 ✓
16,17	1000- ✓
16,24	1-000 ✓
3,19	-0011 ✓
9,13	01-01 D
9,25	-1001 ✓
17,19	100-1 ✓
17,25	1-001 ✓
24,25	1100- ✓
13,15	011-1 E
14,15	0111- F
19,27	1-011 ✓
25,27	110-1 ✓
15,31	-1111 G
27,31	11-11 H

i	Block
0,1,8,9	0-00- ✓
0,1,16,17	-000- ✓
0,8,16,24	--000 ✓
1,3,17,19	-00-1 C
1,9,17,25	--001 ✓
8,9,24,25	-100- ✓
16,17,24,25	1-00- ✓
17,19,25,27	1-0-1 B

i	Block
0,1,8,9,16,17,24,25	--00- A

Ergebnis:

A: $\bar{c} \cdot \bar{b}$ B: $e \cdot \bar{c} \cdot a$ C: $\bar{d} \cdot \bar{c} \cdot a$ D: $\bar{e} \cdot d \cdot \bar{b} \cdot a$
 E: $\bar{e} \cdot d \cdot c \cdot a$ F: $\bar{e} \cdot d \cdot c \cdot b$ G: $d \cdot c \cdot b \cdot a$ H: $e \cdot d \cdot b \cdot a$

4.7.3 Verfahren nach Nelson

Das Nelson-Verfahren ist ein sehr leistungsfähiges algebraisches Verfahren zur Bestimmung der Menge aller Primimplikanten bzw. Primimplikate.

Bei der Bestimmung der **Primimplikanten** wird beliebige konjunktive Form einer SF aufgestellt, und durch Anwenden von Distributiv- und

Absorptionsgesetz umgeformt in eine disjunktive Form. Diese disjunktive Form enthält:

- nur Primimplikanten
- alle Primimplikanten

Bei unvollständigen SFs ist der Ausgangspunkt eine konjunktive Form, die die Freistellen nicht enthält. Im Ergebnis sind dann - wie erforderlich - die Freistellen enthalten („subtraktives Verfahren“).

Das Nelson-Verfahren zur Bestimmung aller **Primimplikanten (PI)** einer Schaltfunktion f kann wie folgt formuliert werden:

1. Ausgangspunkt ist eine beliebige konjunktive Form von f , die keine Redundanzen enthält (d. h. eine konjunktive Form der Einsvervollständigung von f).
2. Diese konjunktive Form wird durch Anwenden des Distributivgesetzes und des Absorptionsgesetzes umgeformt in eine disjunktive Form.
3. Die entstandene disjunktive Form enthält nur Primimplikanten von f und sie enthält alle Primimplikanten von f .

Entsprechend lässt sich das Nelson-Verfahren zur Bestimmung aller **Primimplikate (PA)** einer Schaltfunktion f wie folgt formulieren:

1. Ausgangspunkt ist eine beliebige disjunktive Form von f , die keine Redundanzen enthält (d. h. eine disjunktive Form der Nullvervollständigung von f).
2. Diese disjunktive Form wird durch Anwenden des Distributivgesetzes und des Absorptionsgesetzes umgeformt in eine konjunktive Form.
3. Die entstandene konjunktive Form enthält nur Primimplikate von f und sie enthält alle Primimplikate von f .

Beispiel 1: Gesucht seien alle PIs der folgenden SF

<u>a</u>			
0	1	1	0
0	0	1	1
0	0	1	-
-	1	0	0
<u>c</u>			

b

d

Ausgangspunkt: konjunktive Form der Einsvervollständigung

$$\begin{aligned}
 f &= (\bar{d} \vee \bar{c} \vee b) (c \vee \bar{b}) (d \vee b \vee a) \\
 &= (c \vee \bar{b}) (\bar{d} \vee \bar{c} \vee b) (d \vee b \vee a) \\
 &= (c \vee \bar{b}) (0 \vee \bar{d} \vee b \vee \bar{d} \vee a \vee d \vee \bar{c} \vee \bar{c} \vee b \vee \bar{c} \vee a \vee d \vee b \vee b \vee b \vee a) \\
 &= (c \vee \bar{b}) (\bar{d} \vee a \vee d \vee \bar{c} \vee \bar{c} \vee a \vee b) \\
 &= \bar{d} c a \vee 0 \vee 0 \vee c b \vee \bar{d} \bar{b} a \vee d \bar{b} \bar{c} \vee \bar{c} \bar{b} a \vee 0 \\
 &= \bar{d} c a \vee c b \vee \bar{d} \bar{b} a \vee d \bar{b} \bar{c} \vee \bar{c} \bar{b} a
 \end{aligned}$$

Ergebnis:

$$\begin{aligned}
 &c b \\
 &\bar{d} c a \\
 &\bar{d} \bar{b} a \\
 &\bar{c} \bar{b} a \\
 &d \bar{c} \bar{b}
 \end{aligned}$$

Beispiel 2: Gesucht seien alle PAs der folgenden SF

<u>a</u>			
0 ₀	1 ₁	1 ₅	0 ₄
0 ₂	0 ₃	1 ₇	1 ₆
0 ₁₀	0 ₁₁	1 ₁₅	1 ₁₄
0 ₈	1 ₉	0 ₁₃	0 ₁₂
<u>c</u>			

b

d

Ausgangspunkt: disjunktive Form

$$\begin{aligned}
 f &= cb \vee \bar{d}ca \vee \bar{c}\bar{b}a \\
 &= [(c \vee \bar{d}) \cdot (c) \cdot (c \vee a) \cdot (\bar{d} \vee b) \cdot (c \vee b) \cdot (b \vee a)] \vee [\bar{c} \cdot \bar{b} \cdot a] \\
 &= [(c) \cdot (\bar{d} \vee b) \cdot (b \vee a)] \vee [\bar{c} \cdot \bar{b} \cdot a] \\
 &= (1) \cdot (c \vee \bar{b}) \cdot (c \vee a) \cdot (\bar{d} \vee \bar{c} \vee b) \cdot (1) \cdot (\bar{d} \vee b \vee a) \cdot (\bar{c} \vee b \vee a) \cdot (1) \cdot (b \vee a) \\
 &= (c \vee \bar{b}) \cdot (c \vee a) \cdot (b \vee a) \cdot (\bar{d} \vee \bar{c} \vee b)
 \end{aligned}$$

$$\text{Ergebnis: } PA_1 = c \vee \bar{b} \quad PA_2 = c \vee a \quad PA_3 = b \vee a \quad PA_4 = \bar{d} \vee \bar{c} \vee b$$

4.8 Disjunktive und konjunktive Minimalform

Um eine Schaltfunktion f mit minimalem Aufwand zu realisieren, werden in der Regel nicht alle Primimplikanten bzw. Primimplikate benötigt. Es ist also eine geeignete Auswahl zu treffen, d. h. eine Teilmenge zu bestimmen.

Die Auswahl ist so zu treffen, dass mit den verbleibenden PIs alle Einsstellen von f bzw. mit den verbleibenden PAs alle Nullstellen von f erfasst ("überdeckt") werden.

Im Folgenden werden die Begriffe „redundante/irredundante Überdeckung“ für den disjunktiven Fall definiert. Die Definition für den konjunktiven Fall erfolgt analog.

Definition (Überdeckung):

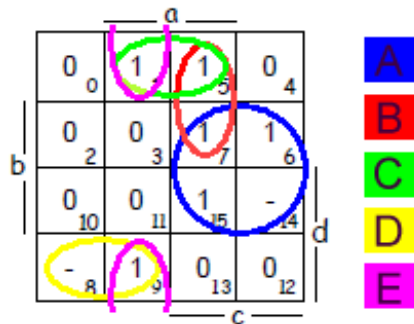
Eine (echte oder unechte) Teilmenge aller PIs einer Schaltfunktion f heißt Überdeckung von f , wenn **alle** Einsstellen von f in den zugehörigen Primeinsblöcken enthalten sind.

Die Funktion f lässt sich dann darstellen als disjunktive Verknüpfung der zu der Überdeckung gehörenden PIs.

Definition (redundante/irredundante Überdeckung):

Eine Überdeckung heißt redundant, wenn ein PI entfernt werden kann, und die verbleibende Teilmenge von PIs immer noch eine Überdeckung darstellt. Ansonsten handelt es sich um eine irredundante (nicht-redundante) Überdeckung.

Beispiel: SF $f(d,c,b,a)$



Betrachten wir die folgenden Teilmengen von PIs:

TM1 = {A, B, C, D, E}

TM2 = {A, C, D, E}

TM3 = {A, C, E}

TM4 = {B, C, D, E}

TM4 ist keine Überdeckung, da die Einsstelle bei B_6 (und auch die Einsstelle bei B_{15}) nicht enthalten ist („überdeckt ist“).

Alle anderen 3 Teilmengen sind Überdeckungen, wobei TM1 und TM2 noch Redundanz enthalten. TM3 ist eine irredundante Überdeckung.

Eine kostenminimale Realisierung einer SF basiert immer auf einer irredundanten Überdeckung.

Definition (Kern-Primimplikant):

Ein PI heißt Kern-PI, wenn er in jeder Überdeckung vorkommen muss.

Dies ist der Fall, wenn eine Einsstelle der SF nur in einem einzigen Primeinsblock liegt. Im obigen Beispiel ist das für die Einsstellen bei B_6 und B_{15} der Fall, die nur vom PI A überdeckt werden. Damit ist der PI A ein Kern-Primimplikant.

Die Überdeckungstabelle beschreibt formal, welche PIs welche Einstellen überdecken. Sie ist wie folgt aufgebaut:

PI	1	5	6	7	9	15
A			x	x		x
B		x		x		
C	x	x				
D					x	
E	x				x	

Ein Kern-PI ist daran zu erkennen, dass es eine Spalte mit nur einem Kreuz gibt. Sind Kern-PIs vorhanden, so erhält man die reduzierte Überdeckungstabelle durch Streichen der Zeilen mit Kern-PIs und durch Streichen der Spalten mit Einstellen, die durch die Kern-PIs erfasst sind.

Die reduzierte Überdeckungstabelle für obiges Beispiel sieht dann wie folgt aus:

PI	1	5	9
B		x	
C	x	x	
D			x
E	x		x

Verfahren zur Bestimmung aller irredundanten Überdeckungen

Ein algebraisches Verfahren zur Bestimmung aller irredundanten Überdeckungen ist das **Petrick-Verfahren**. Zur Minimierung des Rechenaufwands wird es üblicherweise auf die reduzierte Überdeckungstabelle angewandt: in diesem Fall müssen anschließend noch die Kern-PIs dem Ergebnis angefügt werden.

Dem Petrick-Verfahren liegt die folgende Idee zu Grunde: Man stellt zur Beschreibung der Überdeckungsverhältnisse einen schaltalgebraischen Ausdruck auf, den sogenannte Petrick-Ausdruck P . Er ist eine logische Bedingung dafür, dass jede Einstelle berücksichtigt wurde:

$$P = (1.\text{Einstelle überdeckt}) \& (2.\text{ Einsts. überd.}) \& \dots \& (n\text{-te Einsts. überd.})$$

Ist $P = 1$, so sind alle Einstellen erfasst, d. h. die Überdeckungseigenschaft erfüllt.

Den Primimplikanten werden so genannte Präsenzvariablen zugeordnet gemäß dem folgenden Schema:

$p_A = 1$: Primimplikant A ist in der Auswahl vorhanden.

$p_A = 0$: Primimplikant A ist in der Auswahl nicht vorhanden.

Mit der Einführung der Präsenzvariablen kann nun die Information einer Überdeckungstabelle spaltenweise in den Petrick-Ausdruck übertragen werden. Das Vorgehen wird aus dem Beispiel der obigen reduzierten Überdeckungstabelle deutlich: Hier ergibt sich der folgende Petrick-Ausdruck:

$P = (\text{Einsst. bei } B_1 \text{ überd.}) \& (\text{Einsst. bei } B_5 \text{ überd.}) \& (\text{Einsst. bei } B_9 \text{ überd.})$

$$P = (p_C \vee p_E) \cdot (p_B \vee p_C) \cdot (p_D \vee p_E)$$

Der Petrick-Ausdruck liegt also zunächst in konjunktiver Form vor und wird nun – wie beim Nelson-Verfahren (Distributivgesetz, Absorptionsgesetz) – in einen disjunktiven Ausdruck umgeformt.

In unserem Beispiel erhält man:

$$P = p_C \cdot p_D \vee p_C \cdot p_E \vee p_B \cdot p_E$$

Da $P=1$ gelten muss, kann man hieraus folgern, dass die Teilmengen $\{C, D\}$ oder $\{C, E\}$ oder $\{B, E\}$ jeweils ausreichen, um alle Einstellen zu überdecken. Somit gibt es 3 irredundante Überdeckungen. Die Anwendung des Absorptionsgesetzes hat im Übrigen dafür gesorgt, dass redundante Überdeckungen im Ergebnis nicht auftauchen.

Man muss jetzt noch den Kern-PI A hinzufügen und erhält das folgende Ergebnis (IU=irredundante Überdeckung):

$$IU1 = \{A, C, D\}$$

$$IU2 = \{A, C, E\}$$

$$IU3 = \{A, B, E\}$$

Jede Überdeckung entspricht nun einer möglichen Realisierung der Schaltfunktion:

$$R1: f(d, c, b, a) = c \cdot b \vee \bar{d} \cdot \bar{b} \cdot a \vee d \cdot \bar{c} \cdot \bar{b}$$

$$R2: f(d, c, b, a) = c \cdot b \vee \bar{d} \cdot \bar{b} \cdot a \vee \bar{c} \cdot \bar{b} \cdot a$$

$$R3: f(d, c, b, a) = c \cdot b \vee \bar{d} \cdot c \cdot a \vee \bar{c} \cdot \bar{b} \cdot a$$

Kostenminimale Realisierung einer Schaltfunktion

Wie bereits erwähnt, basiert eine kostenminimale Realisierung einer SF immer auf einer irredundanten Überdeckung. Welche der irredundanten Überdeckungen kostenminimal ist (falls es mehrere gibt), hängt davon ab, wie der Realisierungsaufwand bewertet wird. Dazu muss eine Kostenfunktion definiert werden.

Für unser Beispiel sollen zwei verschiedene Kostenfunktionen K_1 und K_2 definiert werden:

K_1 : Jeder Gattereingang in einem UND-Gatter und in dem ODER-Gatter verursacht den Kostenwert 2.

K_2 : Jeder Gattereingang verursacht Kosten wie bei K_1 . Jede Inversion einer Schaltvariablen verursacht zusätzlich den Kostenwert 1.

Damit ergeben sich die folgenden Kosten für die Realisierungen $R_1 - R_3$:

	K_1	K_2
R1	22	26
R2	22	26
R3	22	25

Die kostenminimale Realisierung wird **disjunktive Minimalform (DMF)** der Schaltfunktion genannt. Bezüglich der Kostenfunktion K_1 gibt es 3 DMFs, bezüglich der Kostenfunktion K_2 ist die DMF eindeutig.

DMF bzgl. K_2 : $f(d, c, b, a) = c \cdot b \vee \bar{d} \cdot c \cdot a \vee \bar{c} \cdot \bar{b} \cdot a$

Bei der Bestimmung der **konjunktiven Minimalform (KMF)** einer Schaltfunktion f werden in der Überdeckungstabelle die Primimplikate von f aufgelistet und es wird markiert, welche Nullstellen von f in den zugehörigen Primnullblöcken enthalten sind. Die Überdeckungstabelle beschreibt im konjunktiven Fall, welche PAs welche Nullstellen überdecken.

Das weitere Vorgehen zur Bestimmung der KMF(s) erfolgt auf die gleiche Art und Weise: Bestimmung der Kern-PAs, reduzierte Überdeckungstabelle, Bestimmung aller irredundanten Überdeckungen, Berechnung der Kostenwerte.

4.9 Rechenschaltungen

Für die Addition von Dualzahlen wird für jede Stelle ein Schaltnetz benötigt, das die Werte a und b der Operanden und den Übertrag c_i aus der letzten Stelle verarbeitet. Das Schaltnetz liefert das Ergebnis s für die betrachtete Stelle und den Übertrag c_o für die nächste Stelle. Dieses Schaltnetz wird Volladdierer genannt.

a	0	0	0	1	0
b	0	1	0	1	1
s	0	1	1	0	1

$a=0 \ b=0 \ c_i=1$
 $s=1 \ c_o=0$

In der niederwertigsten Stelle ist kein Übertrag zu berücksichtigen: hierfür würde ein einfacheres Schaltnetz (Halbaddierer) genügen. Oftmals wird auch an dieser

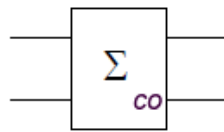
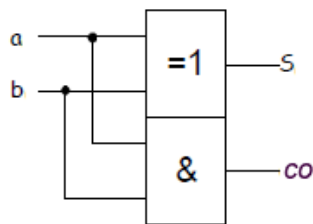
Stelle ein Volladdierer verwendet, bei dem der *ci*-Eingang fest mit 0 verschaltetet ist.

Halbaddierer:

a	b	s	co
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$s = a \oplus b$$

$$co = a \cdot b$$



Volladdierer:

a	b	ci	s	co	Bemerkung
0	0	0	0	0	0 Einsen
0	0	1	1	0	1 Eins
0	1	0	1	0	1 Eins
0	1	1	0	1	2 Einsen
1	0	0	1	0	1 Eins
1	0	1	0	1	2 Einsen
1	1	0	0	1	2 Einsen
1	1	1	1	1	3 Einsen

$$s = a \oplus b \oplus ci$$

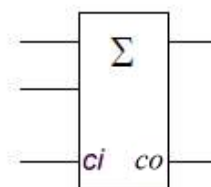
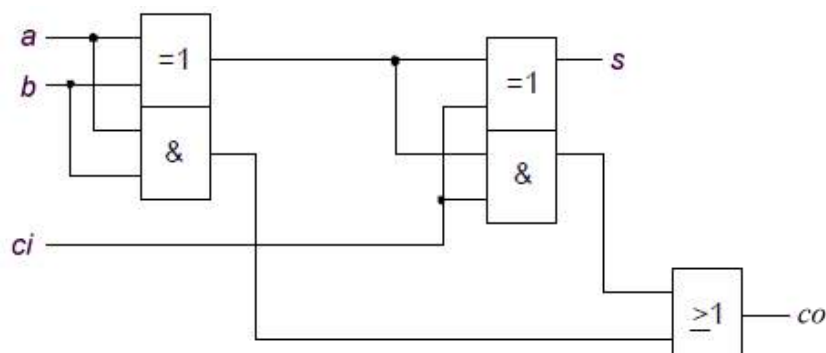
$$co = a \cdot b \vee ci \cdot (a \oplus b)$$

Die Summe *s* ist 1, wenn die Anzahl der Einsen ungerade ist:

$$s = \bar{a} \cdot \bar{b} \cdot ci \vee \bar{a} \cdot b \cdot \bar{ci} \vee a \cdot \bar{b} \cdot \bar{ci} \vee a \cdot b \cdot ci = \bar{ci} \cdot (a \oplus b) \vee ci \cdot a \oplus b = ci \oplus a \oplus b$$

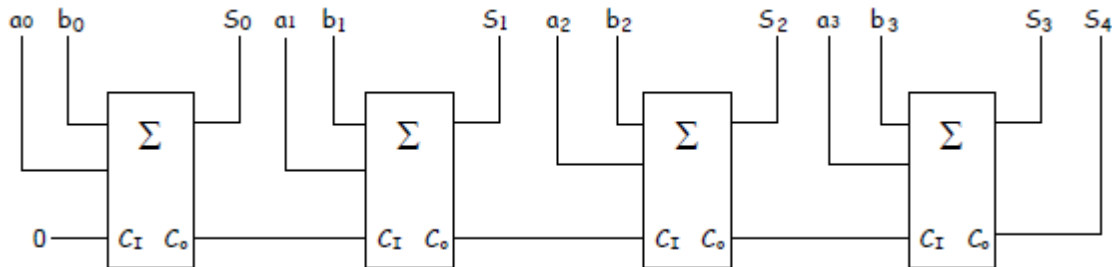
Der Übertrag *co* ist eins, wenn die Anzahl der Einsen 2 oder 3 ist:

$$co = \bar{a} \cdot b \cdot ci \vee a \cdot \bar{b} \cdot ci \vee a \cdot b \cdot \bar{ci} \vee a \cdot b \cdot ci = ci \cdot (a \oplus b) \vee a \cdot b$$



Addierer mit Serienübertrag:

4 Volladdierer können auf die folgende Art zu einem 4-Bit-Addierer zusammengeschaltet werden:



Dieser Addierer mit Serienübertrag (ripple carry adder) hat eine einfache Struktur und ist mit wenig Aufwand realisierbar. Er ist aber nicht geschwindigkeitsoptimal und hat den Nachteil, dass das Ergebnis nicht gleichzeitig an den s-Ausgängen bereitsteht.

Die Ursache liegt darin, dass der *co*-Ausgang von dem *ci*-Eingang abhängt, so dass ein Volladdierer jeweils auf das Ergebnis seines Vorgängers in der Kette warten muss.

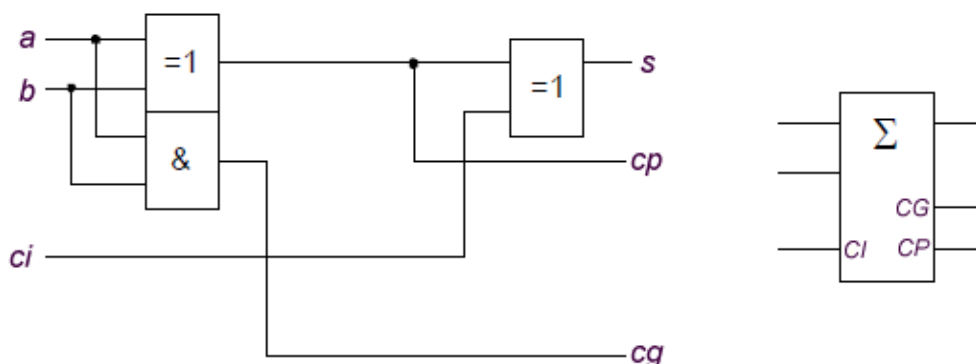
Volladdierer mit carry generate und carry propagate:

Eine modifizierte Version des Volladdierers besitzt statt dem *co*-Ausgang die Ausgänge *cg* (carry generate) und *cp* (carry propagate). Diese hängen nicht vom dem *ci*-Eingang ab und können daher sofort berechnet werden, wenn die Operanden *a* und *b* anliegen. Die Definition und die Bedeutung dieser Ausgänge ist aus der folgenden Wahrheitstabelle ersichtlich:

a	b	cg	cp	Bemerkung
0	0	0	0	$co=0$
0	1	0	1	$co=1$ falls $ci=1$
1	0	0	1	$co=1$ falls $ci=1$
1	1	1	0	$co=1$

$cp = a \oplus b$

$cg = a \cdot b$



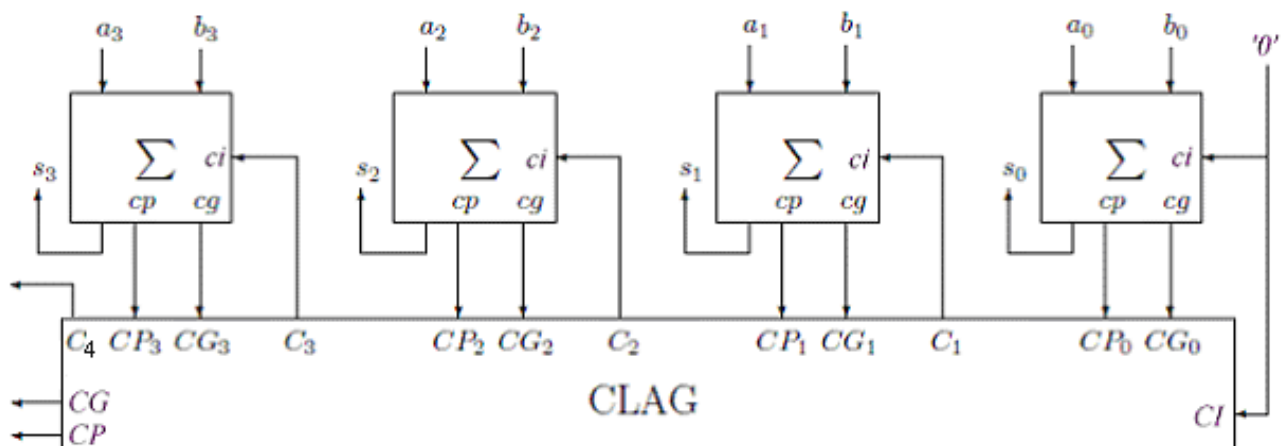
Das *co* könnte nun wie folgt aus *cg* und *cp* berechnet werden:

$$co = cg \vee ci \cdot cp$$

Diese Berechnung übernimmt aber – und das für alle Stellen gleichzeitig – ein weiteres Schaltnetz, der **Carry-Look-Ahead-Generators (CLAG)**. Damit lässt sich ein Addierer mit Parallelübertrag realisieren.

Addierer mit Parallelübertrag:

4 Volladdierer(mit cg und cp) und ein geeigneter CLAG können auf die folgende Art zu einem 4-Bit-Addierer mit Parallelübertrag (Carry-Look-Ahead-Adder) zusammengeschaltet werden:



Für die Ausgänge des CLAG gelten die folgenden logischen Gleichungen:

$$C_1 = CG_0 \vee CI \cdot CP_0$$

$$C_2 = CG_1 \vee CG_0 \cdot CP_1 \vee CI \cdot CP_0 \cdot CP_1$$

$$C_3 = CG_2 \vee CG_1 \cdot CP_2 \vee CG_0 \cdot CP_1 \cdot CP_2 \vee CI \cdot CP_0 \cdot CP_1 \cdot CP_2$$

$$C_4 = CG_3 \vee CG_2 \cdot CP_3 \vee CG_1 \cdot CP_2 \cdot CP_3 \vee CG_0 \cdot CP_1 \cdot CP_2 \cdot CP_3 \\ \vee CI \cdot CP_0 \cdot CP_1 \cdot CP_2 \cdot CP_3$$

Üblicherweise hat ein CLAG-Baustein keinen C_4 -Ausgang (was dem CO-Ausgang entsprechen würde), dafür aber CG - und CP -Ausgänge. Wegen dem Zusammenhang $C_4 = CO = CG \vee CI \cdot CP$ lauten die logischen Gleichungen für die CG - und CP -Ausgänge:

$$CG = CG_3 \vee CG_2 \cdot CP_3 \vee CG_1 \cdot CP_2 \cdot CP_3 \vee CG_0 \cdot CP_1 \cdot CP_2 \cdot CP_3$$

$$CP = CP_0 \cdot CP_1 \cdot CP_2 \cdot CP_3$$

Die CG - und CP -Ausgänge ermöglichen die Kaskadierung von mehreren CLAG-Bausteinen. Ein Beispiel kann der "application section" des folgenden Datenblattauszugs entnommen werden:

Auszüge aus dem Datenblatt eines Carry-Look-Ahead-Generators:

SN54S182, SN74S182 LOOK-AHEAD CARRY GENERATORS

SDLS206 – DECEMBER 1972 – REVISED MARCH 1988

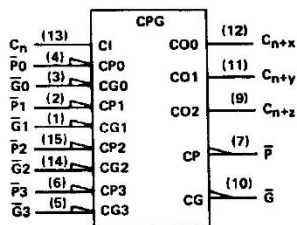
Directly Compatible for Use With:
SN54LS181/SN74LS181,
SN54S281/SN74S281, SN54S381,
SN74S381, SN54S481/SN74S481

PIN DESIGNATIONS

ALTERNATIVE DESIGNATIONS†	PIN NOS.	FUNCTION
$\bar{G}_0, \bar{G}_1, \bar{G}_2, \bar{G}_3$	3, 1, 14, 5	CARRY GENERATE INPUTS
$\bar{P}_0, \bar{P}_1, \bar{P}_2, \bar{P}_3$	4, 2, 15, 6	CARRY PROPAGATE INPUTS
C_n	13	CARRY INPUT
$C_{n+x}, C_{n+y}, C_{n+z}$	12, 11, 9	CARRY OUTPUTS
$\bar{G}$	10	CARRY GENERATE OUTPUT
$\bar{P}$	7	CARRY PROPAGATE OUTPUT
V_{CC}	16	SUPPLY VOLTAGE
GND	8	GROUND

†Interpretations are illustrated in the 'LS181, 'S181 data sheet.

logic symbol‡



‡This symbol is in accordance with ANSI/IEEE Std. 91-1984 and IEC Publication 617-12.
 Pin numbers shown are for D, J, N, and W packages.

description

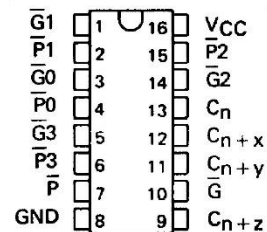
The SN54S182 and SN74S182 are high-speed, look-ahead carry generators capable of anticipating a carry across four binary adders or group of adders. They are cascadable to perform full look-ahead across n-bit adders. Carry, generate-carry, and propagate-carry functions are provided as enumerated in the pin designation table above.

When used in conjunction with the 'LS181 or 'S181 arithmetic logic unit (ALU), these generators provide high-speed carry look-ahead capability for any word length. Each 'S182 generates the look-ahead (anticipated carry) across a group of four ALUs and, in addition, other carry look-ahead circuits may be employed to anticipate carry across sections of four look-ahead packages up to n-bits. The method of cascading 'S182 circuits to perform multilevel look-ahead is illustrated under typical application data.

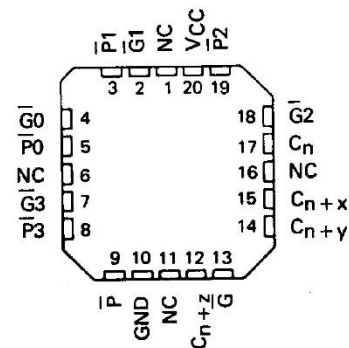
The carry functions (inputs, outputs, generate, and propagate) of the look-ahead generators are implemented in the compatible forms for direct connection to the ALU. Reinterpretations of carry functions as explained on the 'LS181 and 'S181 data sheet are also applicable to and compatible with the look-ahead generator. Logic equations for the 'S182 are:

$$\begin{aligned}
 C_{n+x} &= G_0 + P_0 C_n \\
 C_{n+y} &= G_1 + P_1 G_0 + P_1 P_0 C_n \\
 C_{n+z} &= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_n \\
 \bar{G} &= \bar{G}_3 + P_3 \bar{G}_2 + P_3 P_2 \bar{G}_1 + P_3 P_2 P_1 \bar{G}_0 \\
 \bar{P} &= \bar{P}_3 \bar{P}_2 \bar{P}_1 \bar{P}_0
 \end{aligned}
 \quad \text{or} \quad
 \begin{aligned}
 \bar{C}_{n+x} &= \bar{Y}_0 (X_0 + C_n) \\
 \bar{C}_{n+y} &= \bar{Y}_1 [X_1 + Y_0 (X_0 + C_n)] \\
 \bar{C}_{n+z} &= \bar{Y}_2 \{X_2 + Y_1 [X_1 + Y_0 (X_0 + C_n)]\} \\
 Y &= Y_3 (X_3 + Y_2) (X_3 + X_2 + Y_1) (X_3 + X_2 + X_1 + Y_0) \\
 X &= X_3 + X_2 + X_1 + X_0
 \end{aligned}$$

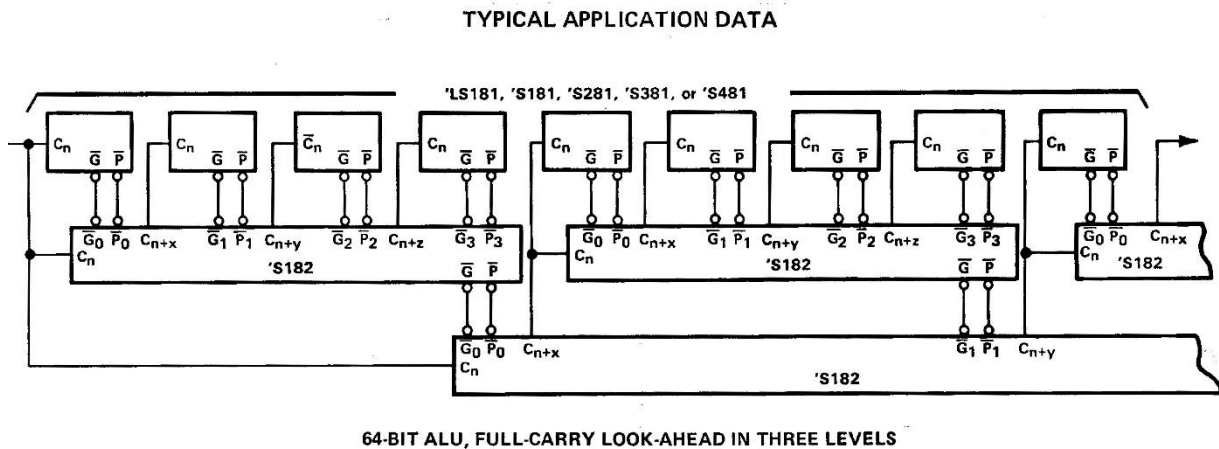
SN54S182 ... J OR W PACKAGE
 SN74S182 ... D OR N PACKAGE
 (TOP VIEW)



SN54S182 ... FK PACKAGE
 (TOP VIEW)



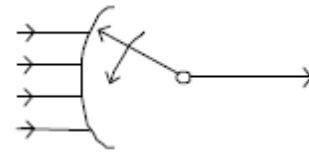
NC - No internal connection



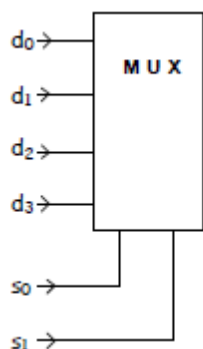
4.10 Multiplexer-Schaltnetze

Multiplexer und Demultiplexer

Ein Multiplexer (MUX) wählt aus mehreren Eingängen einen aus, um ihn auf den Ausgang durchzuschalten. Die Entsprechung in der „analogen Welt“ wäre nebenstehender Umschalter.



In der „digitalen Welt“ geschieht die Auswahl des Eingangs durch zusätzliche Steuereingänge (die Eingangszahl ist daher immer eine 2er-Potenz). Ein 4:1-MUX besitzt z. Bsp. 4 Daten- und 2 Steuereingänge:



Für den Ausgang y gilt die folgende Gleichung:

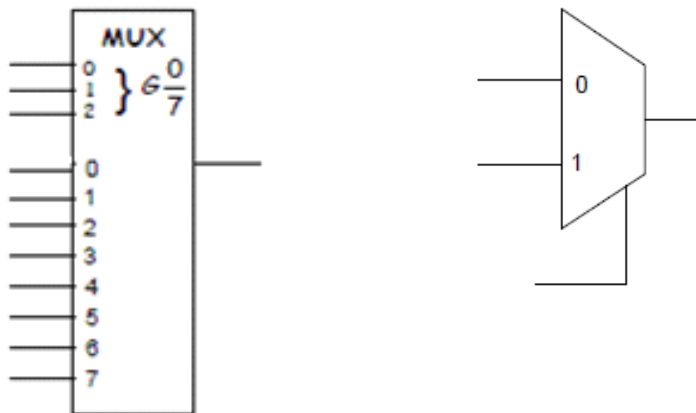
$$y = \overline{s_0} \overline{s_1} d_0 \vee \overline{s_0} s_1 d_1 \vee s_0 \overline{s_1} d_2 \vee s_0 s_1 d_3$$

Für einen $n:1$ -MUX gilt allgemein:

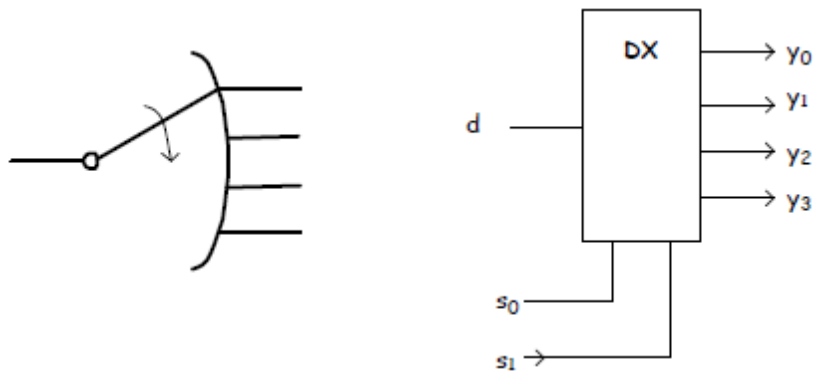
$$y = m_0 \cdot d_0 \vee m_1 \cdot d_1 \vee \dots \vee m_{n-1} \cdot d_{n-1}, \quad (\text{Gl. 4.1})$$

wobei m_j die Minterme der Steuervariablen bezeichnen.

Die folgende Darstellung zeigt das Schaltsymbol für den MUX nach der internationalen Norm am Beispiel eines 8:1-MUX. Daneben ist Symbol für den 2:1-MUX aus der amerikanischen Norm dargestellt, da dieses im Folgenden auch öfters verwendet wird:



Ist ein Dateneingang vorhanden, der wahlweise auf mit einen der Ausgänge verbunden werden kann, so heißt die entsprechende Schaltung Demultiplexer (DEMUX, DX). Auch hierfür gibt es eine Entsprechung aus der „analogen Welt“:



In der „digitalen Welt“ haben alle nicht-ausgewählten Ausgänge einen festen Logikwert, z. Bsp. 0. Dann die gelten folgenden logischen Gleichungen:

$$y_3 = d \, s_1 \, s_0$$

$$y_2 = d \, s_1 \, \overline{s_0}$$

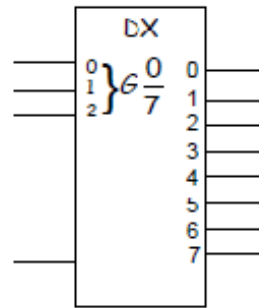
$$y_1 = d \, \overline{s_1} \, s_0$$

$$y_0 = d \, \overline{s_1} \, \overline{s_0}$$

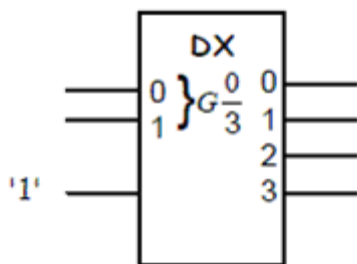
Oder allgemein für einen DEMUX mit n Dateneingängen:

$$y_k = d \cdot m_k \quad \text{für } 0 \leq k \leq n-1.$$

Das Schaltsymbol des DEMUX ist das folgende:

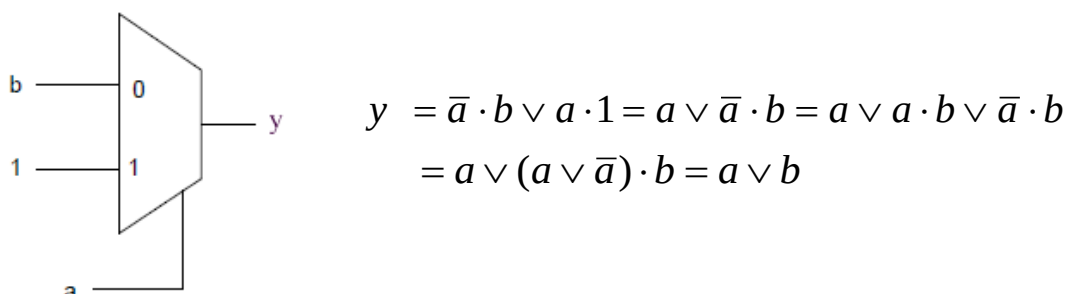
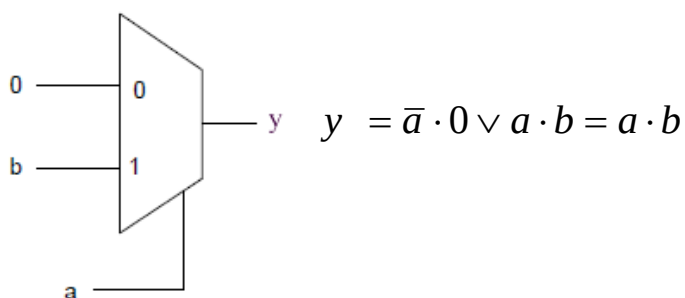


Ein DEMUX mit fest verschaltetem Dateneingang könne auch als Dekoder (z. Bsp. zur Ansteuerung von Speicherbausteinen) verwendet werden:

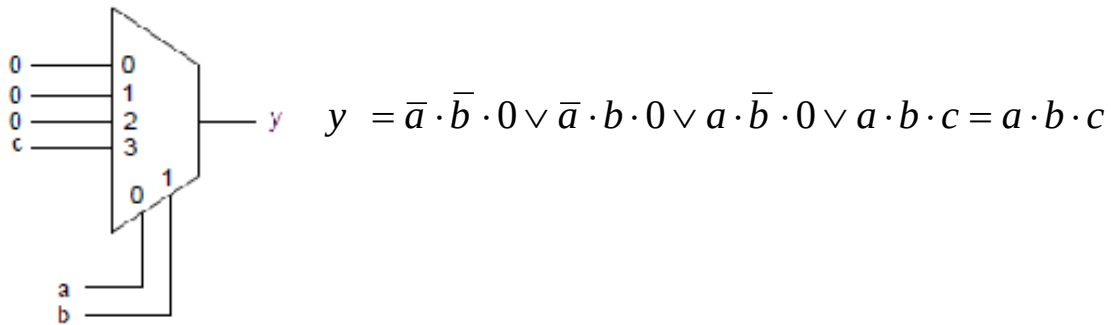


1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

Mit Multiplexern lassen sich beliebige Schaltnetze realisieren, in dem man die Daten- und Steuereingänge in geeigneter Weise mit den Eingangsvariablen der Schaltfunktion beschaltet. Die Synthese vom Multiplexern-Schaltnetzen erfordert als theoretische Grundlage den so genannten Entwicklungssatz, der im Folgenden behandelt wird. Zunächst aber soll beispielhaft gezeigt werden, wie man mit 2:1-MUX die Booleschen Grundverknüpfungen UND bzw. ODER realisieren kann.



Das nächste Beispiel zeigt die Realisierung einer UND-Verknüpfung mit 3 Eingängen mit Hilfe eines 4:1-MUX:



Der Entwicklungssatz

Definition (Restfunktion):

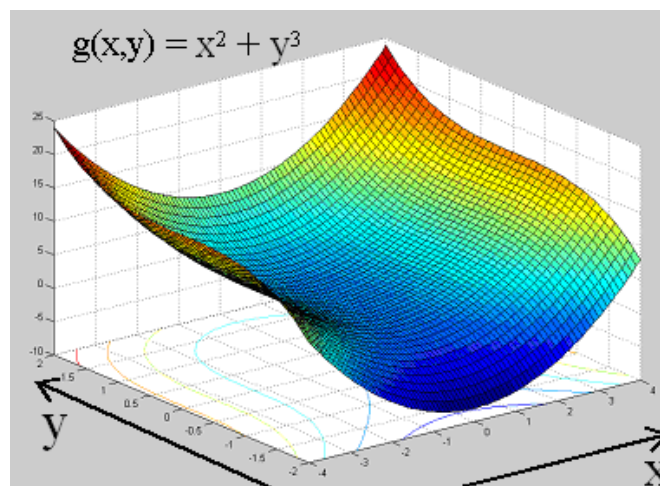
Gegeben sei eine Schaltfunktion $f(x_n, \dots, x_1)$. Setzt man eine Schaltvariable x_i zu 0 oder zu 1, so erhält man eine Restfunktion, die dann nur noch von $n-1$ Schaltvariablen abhängt.

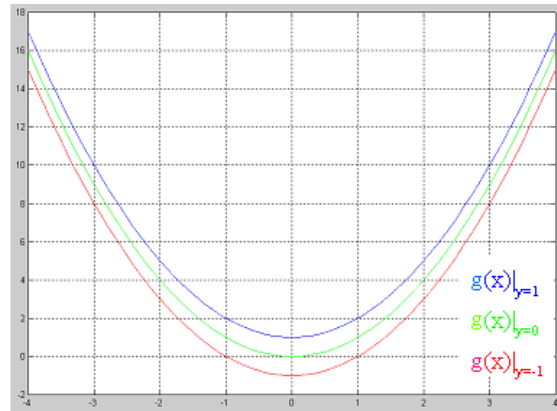
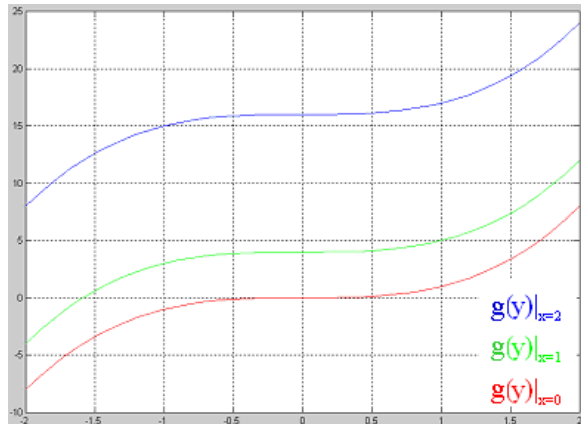
$$f|_{x_i=0}(x_n, \dots, x_{i+1}, x_{i-1}, \dots, x_0) = f(x_n, \dots, x_{i+1}, 0, x_{i-1}, \dots, x_0)$$

$$f|_{x_i=1}(x_n, \dots, x_{i+1}, x_{i-1}, \dots, x_0) = f(x_n, \dots, x_{i+1}, 1, x_{i-1}, \dots, x_0)$$

Der entsprechende Begriff aus der Analysis ist die Schnittfunktion. Als Beispiel werde die Funktion $g(x, y) = x^2 + y^3$ betrachtet. Allerdings sind in der Analysis für jede Variable beliebig viele Schnitte (und nicht nur 2) möglich. Die folgende Darstellung zeigt die Funktion sowie die Schnitte

$$\begin{array}{lll} g|_{y=-1}(x) = x^2 - 1 & g|_{y=0}(x) = x^2 & g|_{y=1}(x) = x^2 + 1 \\ g|_{x=0}(y) = y^3 & g|_{x=1}(y) = y^3 + 1 & g|_{x=2}(y) = y^3 + 4 \end{array}$$





Entwicklungssatz:

Eine Schaltfunktion $f(x_n \dots x_0)$ lässt sich wie folgt nach einer beliebigen Schaltvariablen x_i entwickeln:

$$(1) \quad f(x_n, \dots, x_i, \dots, x_0) = \overline{x_i} \cdot f|_{x_i=0} \vee x_i \cdot f|_{x_i=1}$$

$$(2) \quad f(x_n, \dots, x_i, \dots, x_0) = (\overline{x_i} \vee f|_{x_i=1}) \cdot (x_i \vee f|_{x_i=0})$$

Im Folgenden betrachten wir nur die disjunktive Form (1).

Verbal kann man die disjunktive Form des Entwicklungssatzes wie folgt beschreiben:

Der Boolesche Ausdruck für eine Schaltfunktion lässt sich durch Entwickeln nach einer beliebigen Variablen x_i umformen. Diese Umformung kann verbal wie folgt dargestellt werden:

- (1) Die Entwicklungsvariable x_i wird negiert und mit der Restfunktion $f|_{x_i=0}$ konjunktiv verknüpft;
- (2) die Entwicklungsvariable x_i wird mit der Restfunktion $f|_{x_i=1}$ konjunktiv verknüpft;
- (3) die Produktterme aus (1) und (2) werden disjunktiv miteinander verknüpft.

Satz:

Wird der Boolesche Ausdruck für eine Schaltfunktion nacheinander nach allen Variablen entwickelt (in der Weise, dass immer die entstehenden Restfunktionen nach der nächsten Variablen entwickelt werden), so entsteht die DNF der Schaltfunktion. Die Reihenfolge in der Wahl der Entwicklungsvariablen ist dabei beliebig.

Beispiel:

$$\begin{aligned}
 f(c,b,a) &= (c \oplus b) \oplus a = c \oplus (b \oplus a) = c \oplus b \oplus a \\
 &= \bar{c} \cdot (0 \oplus b \oplus a) \vee c \cdot (1 \oplus b \oplus a) \\
 &= \bar{c} \cdot (\bar{b} \cdot \{0 \oplus 0 \oplus a\} \vee b \cdot \{0 \oplus 1 \oplus a\}) \vee c \cdot (\{1 \oplus 0 \oplus a\} \vee b \cdot \{1 \oplus 1 \oplus a\}) \\
 &= \bar{c} \cdot (\bar{b} \cdot \{\bar{a} \cdot [0 \oplus 0 \oplus 0] \vee a \cdot [0 \oplus 0 \oplus 1]\} \vee b \cdot \{\bar{a} \cdot [0 \oplus 1 \oplus 0] \vee a \cdot [0 \oplus 1 \oplus 1]\}) \\
 &\quad \vee c \cdot (\bar{b} \cdot \{\bar{a} \cdot [1 \oplus 0 \oplus 0] \vee a \cdot [1 \oplus 0 \oplus 1]\} \vee b \cdot \{\bar{a} \cdot [1 \oplus 1 \oplus 0] \vee a \cdot [1 \oplus 1 \oplus 1]\}) \\
 &= \bar{c} \cdot \bar{b} \cdot \bar{a} \cdot [0] \vee \bar{c} \cdot \bar{b} \cdot a \cdot [1] \vee \bar{c} \cdot b \cdot \bar{a} \cdot [1] \vee \bar{c} \cdot b \cdot a \cdot [0] \\
 &\quad \vee c \cdot \bar{b} \cdot \bar{a} \cdot [1] \vee c \cdot \bar{b} \cdot a \cdot [0] \vee c \cdot b \cdot \bar{a} \cdot [0] \vee c \cdot b \cdot a \cdot [1] \\
 &= \bar{c} \cdot \bar{b} \cdot a \vee \bar{c} \cdot b \cdot \bar{a} \vee c \cdot \bar{b} \cdot \bar{a} \vee c \cdot b \cdot a
 \end{aligned}$$

Synthese von Multiplexer-Schaltnetzen:

Soll eine Schaltfunktion mit Hilfe eines Multiplexers realisiert werden, so ist die Vorgehensweise genau so wie im obigen Beispiel (entwickeln nach einer Variable, die Restfunktionen nach der nächsten Variablen entwickeln, usw.) nur dass in der Regel nicht nach allen Variablen entwickelt wird: Die Anzahl der Entwicklungsvariablen muss identisch sein mit der Anzahl der Steuervariablen des gegebenen Multiplexers.

Nach dem Entwickeln (und Ausmultiplizieren) liegt dann die SF in der Form vor, in der sie direkt mit der Multiplexer-Gleichung (Gl. 4.1) verglichen werden kann. Daraus kann dann unmittelbar die geeignete Beschaltung des MUX abgeleitet werden. Es ist offensichtlich, dass die Steuereingänge mit den Entwicklungsvariablen beschaltet werden, die Dateneingänge mit den Restfunktionen.

Es empfiehlt sich, als Entwicklungsvariablen die Variablen auszuwählen, die im Booleschen Ausdruck der SF negiert vorkommen (falls es solche gibt): Die Negation der Steuervariablen ist in der Multiplexerlogik enthalten und verursacht keinen zusätzlichen Aufwand.

Beispiel:

Gegeben sei die Schaltfunktion:

$$y = f(c,b,a) = c \cdot b \vee \bar{c} \cdot a \vee \bar{c} \cdot b \cdot \bar{a}$$

Gesucht sei eine Realisierung mit einem 4:1-Multiplexer.

Vorgehensweise: Entwickeln nach der Variable c und dann nach a :

$$\begin{aligned}
 y &= \bar{c} \cdot (0 \cdot b \vee \bar{0} \cdot a \vee \bar{0} \cdot b \cdot \bar{a}) \vee c \cdot (1 \cdot b \vee \bar{1} \cdot a \vee \bar{1} \cdot b \cdot \bar{a}) \\
 &= \bar{c} \cdot [a \vee b \cdot \bar{a}] \vee c \cdot [b] \\
 &= \bar{c} \cdot [\bar{a} \cdot (0 \vee b \cdot \bar{0}) \vee a \cdot (1 \vee b \cdot \bar{1})] \vee c \cdot [\bar{a} \cdot (b) \vee a \cdot (b)] \\
 &= \bar{c} \cdot [\bar{a} \cdot (b) \vee a \cdot (1)] \vee c \cdot [\bar{a} \cdot (b) \vee a \cdot (b)] \\
 &= \bar{c} \cdot \bar{a} \cdot (b) \vee \bar{c} \cdot a \cdot (1) \vee c \cdot \bar{a} \cdot (b) \vee c \cdot a \cdot (b)
 \end{aligned}$$

